WHOLE-ARRAY RECOMPUTATION OF BORDER ARRAYS: EXACT TWO-CYCLES, SHARP TRANSIENTS, AND FACTORIAL FIBRES

ANONYMOUS

Abstract. Let $\boldsymbol{\beta}(w)$ be the ordinary border array of a finite word. We treat the entire integer array $\boldsymbol{\beta}(w)$ as the next word, recompute all of its borders, and repeat on the inversion-sequence carrier $\mathcal{E}_n = \{(e_0, \ldots, e_{n-1}) : 0 \le e_i \le i\}$. This whole-array recomputation differs from following failure links inside one fixed table. We prove that its one-step image is exactly the valid border arrays. At length $n \ge 2$, the recurrent set consists of $n-1$ explicit cycles of exact period two; length one has one fixed point. An indexed three-state mismatch amplifier gives the sharp maximum transient $0, 0, 1, 2n-4$ for $n = 1, 2, 3, n \ge 4$. We also prove, for every target, the one-step bound $(n-1)!$ and show that equality occurs only at the all-zero table and $(0, 1, 0, \ldots, 0)$ when $n \ge 2$. Border computation, validation, construction, realization, and census are credited background. The owner search is bounded; no novelty, priority, or external-release claim is made.

1. Whole-array recomputation and the subtraction boundary

For a word $w = w_0 \cdots w_{n-1}$ over any alphabet, let

$$\beta_i(w) = \max\{k : 0 \le k \le i,\ w_0 \cdots w_{k-1} = w_{i-k+1} \cdots w_i\}. \tag{1}$$

Thus $\beta_i(w)$ is the longest proper-border length of the prefix ending at i, and $\beta_0(w) = 0$. Write $\boldsymbol{\beta}(w) = (\beta_0(w), \ldots, \beta_{n-1}(w))$. The Morris–Pratt and Knuth–Morris–Pratt mechanisms compute these data in linear time [4]. Validation and realization of border or failure arrays, their generation, and their enumeration have dedicated algorithms and structural theories [1–3]. All of that one-step machinery is background here.

The carrier in this paper is

$$\mathcal{E}_n = \{e = (e_0, \ldots, e_{n-1}) : 0 \le e_i \le i\}. \tag{2}$$

Regarding e as an integer word, define the self-map

$$\Pi_n(e) = \boldsymbol{\beta}(e). \tag{3}$$

It is closed on $\mathcal{E}_n$ because every proper border of an $(i+1)$-letter prefix has length at most i.

The term *whole-array recomputation* prevents a semantic collision. For a fixed word w, the usual failure map sends a positive border length k to $\beta_{k-1}(w)$; iterating that scalar map follows the nested borders of the same fixed word. In (3), by contrast, the complete table $\boldsymbol{\beta}(e)$ becomes a new word:

$$e \longmapsto \boldsymbol{\beta}(e) \longmapsto \boldsymbol{\beta}(\boldsymbol{\beta}(e)) \longmapsto \cdots .$$

No unqualified "iteration of the prefix function" is meant below.

A state is *recurrent* if it belongs to a directed cycle of Π_n. Its depth is

$$\operatorname{depth}(e) = \min\{t \ge 0 : \Pi_n^t(e) \text{ is recurrent}\}.$$

2. The one-step image and canonical two-cycles

Call an integer array *valid* if it is the border array of some word.

Proposition 2.1 (exact image)**.** *The image* $\operatorname{im}\Pi_n$ *is exactly the valid border arrays of length* n*.*

Proof. Every output of Π_n is valid by definition. Conversely, let $p = \boldsymbol{\beta}(w)$ be valid. Replace the first distinct letter of w by zero, the next previously unseen letter by one, and so on. The standardized letter at position i is at most i, so the standardized word $\widehat{w}$ belongs to $\mathcal{E}_n$. Standardization preserves every equality and inequality between positions. Borders depend only on this equality pattern; hence $\boldsymbol{\beta}(\widehat{w}) = \boldsymbol{\beta}(w) = p$. □

2020 *Mathematics Subject Classification.* 37P25, 68R15, 05A05.

Key words and phrases. border array, prefix function, whole-array recomputation, finite dynamical system, inversion sequence, fibre.

For $1 \le r < n$, define

$$A_r = (0, 1, \dots, r, 0, \dots, 0), \tag{4}$$

$$B_{r+1} = (\underbrace{0, \dots, 0}_{r+1}, 1, \dots, 1). \tag{5}$$

Proposition 2.2 (canonical pairs)**.** *For $1 \le r < n$,*

$$\Pi_n(A_r) = B_{r+1}, \qquad \Pi_n(B_{r+1}) = A_r. \tag{6}$$

In particular, these states form $n-1$ distinct cycles of exact period two.

Proof. Along the distinct initial slope of A_r, no nonempty border exists, so the first $r+1$ entries of $\boldsymbol{\beta}(A_r)$ are zero. Every later prefix ends in zero and has the one-letter border 0. If a proper suffix of length at least two were also a prefix, its first letter would force it to start at a tail zero; its second letter would then be zero, contradicting the second prefix letter 1. Thus $\boldsymbol{\beta}(A_r) = B_{r+1}$.

Inside the initial zero run of B_{r+1}, the longest proper border is one shorter than the prefix, producing $0, 1, \dots, r$. Consider a prefix that ends in the one run. A nonempty border must have length greater than $r+1$ so that its prefix copy also ends in one. Its suffix copy must start at a zero, but any proper such start lies inside the initial zero run and gives strictly fewer leading zeros than the prefix copy. Hence no nonempty border exists after the zero run, $\boldsymbol{\beta}(B_{r+1}) = A_r$, and the templates are distinct. □

The remaining task is to prove that no other recurrent state exists and to measure how quickly every state enters one of these pairs.

3. The indexed mismatch amplifier

Every border array p satisfies the unit-growth inequality

$$p_i \le p_{i-1} + 1. \tag{7}$$

For a valid p of length at least two, choose its canonical template as follows. If $p_1 = 1$, let r be maximal such that $p_0 p_1 \cdots p_r = 01 \cdots r$ and put $Q(p) = A_r$. If the slope stops, a realizing word has begun with $r+1$ equal letters. Its next letter either continues that equality and the slope, or differs and destroys every positive border; maximality therefore forces $p_{r+1} = 0$.

If $p_1 = 0$, let k be the length of its maximal initial zero run and put $Q(p) = B_k$. If the run stops, (7) forces $p_k = 1$. Therefore every noncanonical valid table agrees with its template through at least three coordinates. Define its first mismatch index by

$$L(p) = \min\{i : p_i \ne Q(p)_i\}, \tag{8}$$

and put $L(p) = n$ when $p = Q(p)$. The number $L(p)$ is also the length of the agreement prefix.

Lemma 3.1 (indexed mismatch transitions)**.** *Let p be valid, $q = \Pi_n(p)$, and $L = L(p) < n$.*

(i) If $Q(p) = A_r$, then $p_L = 1$, $Q(q) = B_{r+1}$, $L(q) = L$, and $q_L = 2$.

(ii) If $Q(p) = B_k$, then $p_L \in \{0, 2\}$ and $Q(q) = A_{k-1}$. For $p_L = 0$, one has $L(q) = L$ and $q_L = 1$; for $p_L = 2$, one has $L(q) > L$.

(iii) After the extension in (ii), if another mismatch exists, it is an A-type mismatch whose actual value is one.

Equivalently, the first-mismatch states obey

$$A1 \longrightarrow B2 \longrightarrow \textit{extension}, \qquad B0 \longrightarrow A1 \longrightarrow B2 \longrightarrow \textit{extension}. \tag{9}$$

Proof. The border array of a prefix depends only on that prefix. Thus q agrees with $\Pi_n(Q(p))$, the partner in (6), before index L. If $Q(p) = A_r$, then L lies after the first template-tail zero, so this agreement contains the $r+1$ initial zeros of B_{r+1} and its first following one; hence $Q(q) = B_{r+1}$. If $Q(p) = B_k$, then L lies after the first one following the zero run, so the shared prefix contains the complete initial slope of A_{k-1} and its first following zero; hence $Q(q) = A_{k-1}$. It is this complete shared prefix, not the second coordinate alone, that fixes the partner parameter.

Suppose first that $Q(p) = A_r$. The mismatch occurs after the first zero following the slope, so $p_{L-1} = 0$. Its template value is zero; validity, (7), and mismatch force $p_L = 1$. As an integer word, the prefix $p_0 \cdots p_{L-1}$ has border length one. The new letter $p_L = 1$ matches $p_1 = 1$, so the border extends to length two. Hence $q_L = 2$ while the B_{r+1} template has value one, proving (i).

Now suppose $Q(p) = B_k$. Here $p_{L-1} = 1$ and the template value at L is one. Inequality (7) leaves precisely $p_L = 0$ or $p_L = 2$. The integer-word prefix $p_0 \cdots p_{L-1}$ has border length zero. If $p_L = 0$, it matches $p_0 = 0$ and creates border one, which is the $A1$ state. If $p_L = 2$, it does not match p_0, so $q_L = 0$, the required A_{k-1} value; agreement strictly extends. Any later mismatch is now in an A-template tail

after a zero. The preceding A_r-case argument forces its actual value to be one. This proves (ii), (iii), and (9). □

Theorem 3.2 (recurrent atlas and upper clock)**.** *At length one,* (0) *is the unique recurrent state and is fixed. For $n \geq 2$, the recurrent set is exactly*

$$\{A_r, B_{r+1} : 1 \leq r < n\},$$

so it consists of the $n-1$ exact two-cycles in proposition 2.2. For $n \geq 4$,

$$\max_{p \in \operatorname{im} \Pi_n} \operatorname{depth}(p) \leq 2n-5, \qquad \max_{e \in \mathcal{E}_n} \operatorname{depth}(e) \leq 2n-4. \tag{10}$$

Proof. A noncanonical valid table has $L \geq 3$. By lemma 3.1, its first mismatch is passed in at most three updates. After that extension, each later mismatch is passed in at most two updates. Since every extension increases L by at least one, the number of updates before $L = n$ is at most

$$3 + 2(n - L - 1) \leq 3 + 2(n-4) = 2n-5.$$

At $L = n$ the table is one of the canonical templates and is recurrent. Strict growth of L excludes recurrence before that time. Every state of $\mathcal{E}_n$ enters the valid image after one update by proposition 2.1, giving the second bound. The length-one statement and proposition 2.2 finish the recurrent classification. □

4. Sharp trajectories and the small boundaries

For $n \geq 4$, put

$$p_n = (0,0,1,0^{n-3}), \qquad e_n = (0,1,0,2,1^{n-4}), \tag{11}$$

$$X_j = (0,0,1^j,2,0^{n-j-3}) \qquad (1 \leq j \leq n-3), \tag{12}$$

$$Y_j = (0,1,0^{j+1},1,2^{n-j-4}) \qquad (0 \leq j \leq n-4). \tag{13}$$

Proposition 4.1 (sharp witness trajectory)**.** *The following identities hold:*

$$\Pi_n(e_n) = p_n, \qquad \Pi_n(p_n) = Y_0, \tag{14}$$

$$\Pi_n(Y_j) = X_{j+1} \qquad (0 \leq j \leq n-4), \tag{15}$$

$$\Pi_n(X_j) = Y_j \qquad (1 \leq j \leq n-4), \tag{16}$$

$$\Pi_n(X_{n-3}) = A_1. \tag{17}$$

Consequently $\operatorname{depth}(p_n) = 2n-5$ *and* $\operatorname{depth}(e_n) = 2n-4$.

Proof. All identities follow from equality blocks, which we spell out to fix the endpoints. In X_j, the initial 00 creates border values 0, 1; the following j ones and the letter 2 have border zero. If a zero tail is present, its first letter has border one and every later zero has border two. This is Y_j. When $j = n-3$, there is no zero tail, and the output is $A_1 = (0,1,0^{n-2})$.

In Y_j, the second letter differs from the initial zero, giving border zero. Each of the next $j+1$ zeros has the one-letter border 0 and no longer border. The following one completes the border 01 of length two. For a prefix ending in the trailing 2-run, a proper border would contain fewer terminal twos in the initial copy than in the suffix; equality of those counts occurs only for the full, nonproper prefix. Thus every trailing 2 has border zero. The result is X_{j+1}. The same inspection gives the two initial identities in (14).

Thus the orbit of p_n visits $Y_0, X_1, Y_1, X_2, \ldots, X_{n-3}, A_1$ at successive times and first reaches the canonical two-cycle after $2n-5$ updates. None of the preceding states is canonical. Since e_n maps to p_n, it has one additional transient step. □

Theorem 4.2 (piecewise sharp maximum)**.** *For every $n \geq 1$,*

$$\max_{e \in \mathcal{E}_n} \operatorname{depth}(e) = \begin{cases} 0, & n = 1, \\ 0, & n = 2, \\ 1, & n = 3, \\ 2n-4, & n \geq 4. \end{cases} \tag{18}$$

Proof. For $n \geq 4$, theorem 3.2 and proposition 4.1 give matching bounds. At length one there is only the fixed state. At length two the two carrier states are A_1 and B_2, hence both are recurrent. At length three, the four templates are recurrent, while the remaining two states $(0,0,2)$ and $(0,1,1)$ enter a template in one update. This proves every boundary in (18). □

5. Every target fibre and its two unique maxima

The next theorem applies to every target $p \in \mathcal{E}_n$, whether or not p is a valid border array.

Theorem 5.1 (factorial fibre extremum)**.** *For every $p \in \mathcal{E}_n$,*

$$|\Pi_n^{-1}(p)| \leq (n-1)!. \tag{19}$$

When $n \geq 2$, equality holds exactly for

$$p = 0^n \quad \text{and} \quad p = A_1 = (0, 1, 0^{n-2}). \tag{20}$$

At $n = 1$, the sole fibre has size one.

Proof. Expose a source $e \in \mathcal{E}_n$ from left to right. Its first letter is $e_0 = 0$. The prescribed value p_1 uniquely determines e_1: value one requires $e_1 = 0$, and value zero requires $e_1 = 1$. At position $i \geq 2$, a positive prescribed border $p_i = k$ forces the last source letter to match the last letter of the corresponding prefix, namely $e_i = e_{k-1}$; it leaves at most one choice. If $p_i = 0$, then at least $e_i \neq e_0 = 0$, leaving at most the i choices $1, \ldots, i$. Multiplication proves (19); an invalid target simply has no sources.

Equality in the product requires $p_i = 0$ at every $i \geq 2$, because replacing the factor $i \geq 2$ by at most one is strict. Since $p_1 \in \{0, 1\}$, only the two targets in (20) remain. We now prove that both attain the bound without using the false general shortcut that a nonzero final letter alone forbids every longer border.

When $n = 2$, the product over $i = 2, \ldots, n-1$ is empty and equals one: the sources $(0, 1)$ and $(0, 0)$ map respectively to 0^2 and A_1. Hence assume $n \geq 3$ in the suffix arguments that follow.

For target 0^n, choose $e_1 = 1$ and choose each later e_i arbitrarily in $\{1, \ldots, i\}$. Position zero is the only zero. Every proper suffix of a prefix therefore starts with a nonzero letter and cannot equal a prefix, which starts with zero. All border values are zero, and there are $\prod_{i=2}^{n-1} i = (n-1)!$ sources.

For target A_1, choose $e_1 = 0$ and again choose $e_i \in \{1, \ldots, i\}$ for $i \geq 2$. The prefix of length two has border one. For any longer prefix, a proper suffix starting at position at least two fails in its first letter. The only other possible proper suffix starts at position one; its second letter is $e_2 \neq 0 = e_1$, so it also fails. Hence all later border values are zero, giving another $(n-1)!$ sources. These are the only equality cases. For $n = 1$, both the carrier and its image consist of (0). □

6. Exact control, scope, and limitations

The paper-local verifier is deterministic, dependency-free, and uses exact Python integers. It exhausts all

$$\sum_{n=1}^{9} n! = 409{,}113$$

carrier states and the same number of target cells. Through length eight it compares the linear border routine with literal longest-prefix and longest-suffix comparison. It checks every recurrent state and period, every instance of lemma 3.1, both depth bounds, the displayed sharp trajectory, every target fibre, and both unique maximizers. An independent standardization check covers 3,279 words, and the sharp witnesses are continued through every length $4 \leq n \leq 32$. The frozen run makes 1,694,506 exact assertions. These computations search for counterexamples; they do not prove the all-length statements.

The subexceedant carrier, ordinary longest-border convention, and synchronous whole-array recomputation are essential. Strict border arrays, optimized failure functions, a bounded alphabet, or scalar failure-link descent define other systems. The carrier is an inversion-sequence encoding, so generic permutation or factorial language receives no credit; likewise, a sharp clock plus a fibre maximum is only a presentation silhouette. The internal collision controls P105, P112-C6, P122, and the owner-killed same-batch PR1 are therefore excluded from the claimed residual.

The literature search can establish a source hit but cannot certify an absence. The bounded search performed for this project did not locate the whole-array recomputation system or the theorem package above; this non-hit is not novelty or priority evidence. Authorship, posting, submission, specialist contact, and all external-release actions remain on hold.

References

[1] Jean-Pierre Duval, Thierry Lecroq, and Arnaud Lefebvre. Efficient validation and construction of border arrays and validation of string matching automata. *RAIRO – Theoretical Informatics and Applications*, 43(2):281–297, 2009. doi: 10.1051/ita:2008030.

[2] Frantisek Franek, Shudi Gao, Weilin Lu, P. J. Ryan, W. F. Smyth, Yu Sun, and Lu Yang. Verifying a border array in linear time. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 42:223–236, 2002. URL https://www.cas.mcmaster.ca/~franek/proceedings/border.pdf.

[3] Paweł Gawrychowski, Artur Jeż, and Łukasz Jeż. Validating the Knuth–Morris–Pratt failure function, fast and online. *Theory of Computing Systems*, 54(2):337–372, 2014. doi: 10.1007/s00224-013-9522-8.

[4] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977. doi: 10.1137/0206024.